



Chat with Energy Documents

A Retrieval-Augmented Generation (RAG) system that lets engineers and operators query long, technical energy documents in natural language. Project: GAI-13 – Generative AI course, Medicaps University.

Problem Statement

Challenge

Energy documents (manuals, turbine reports, grid specs) are long, technical, and hard to search quickly.

Consequences

Slow manual search, missed critical details, and poor situational response during operations.

Opportunity

Provide context-aware, grounded answers via RAG to speed decisions and reduce errors.





System Overview

The system combines dense retrieval (vector search) with a large language model: retrieve relevant document chunks, then generate answers grounded in those chunks. Two principal phases: Document Ingestion & Indexing, and Context-Aware Question Answering.

High-Level Architecture

Modular, layered design for scalability and testability:

1

Presentation

FastAPI endpoints, web/chat client, authentication

2

Service

Business logic, orchestration, batching

3

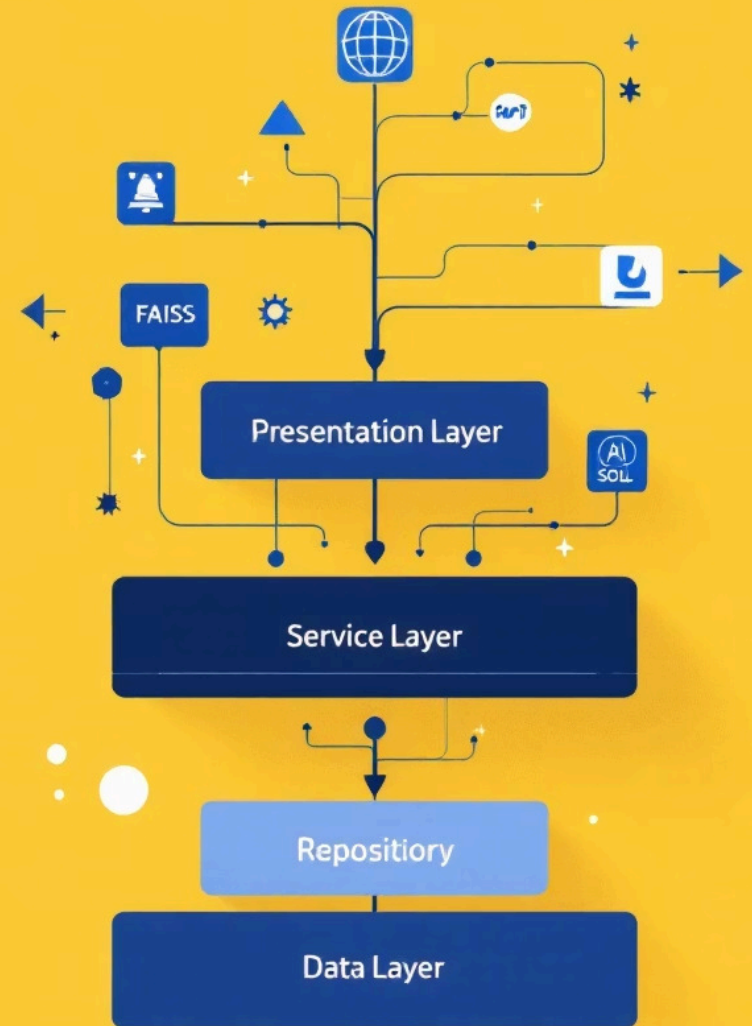
Repository

SQLAlchemy access, metadata, audit trails

4

Data

FAISS vector store + relational DB for metadata





Document Upload Workflow

01

1. Upload & Validate

File checks (type, size), virus scan, secure storage.

02

2. Extract & Clean

OCR / text extraction, normalization, remove noise.

03

3. Chunk & Embed

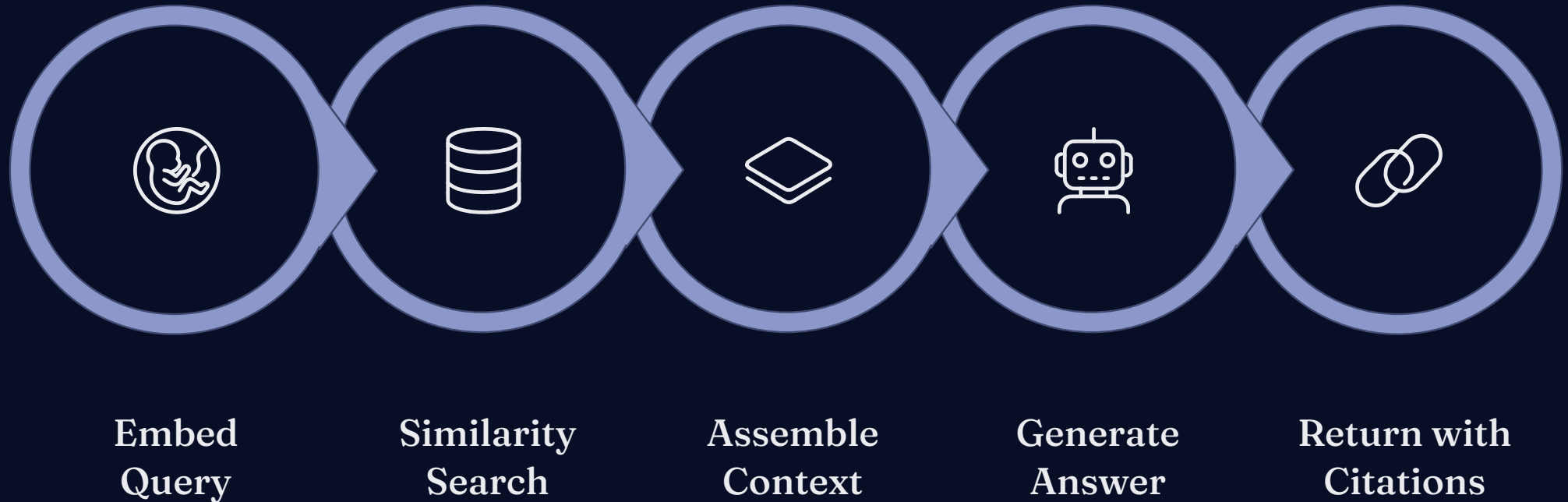
Overlapping chunks, embedding generation (OpenAI-style), batch processing.

04

4. Index & Persist

Store vectors in FAISS; metadata, provenance, and chunk references in SQL DB.

Question → Answer Flow



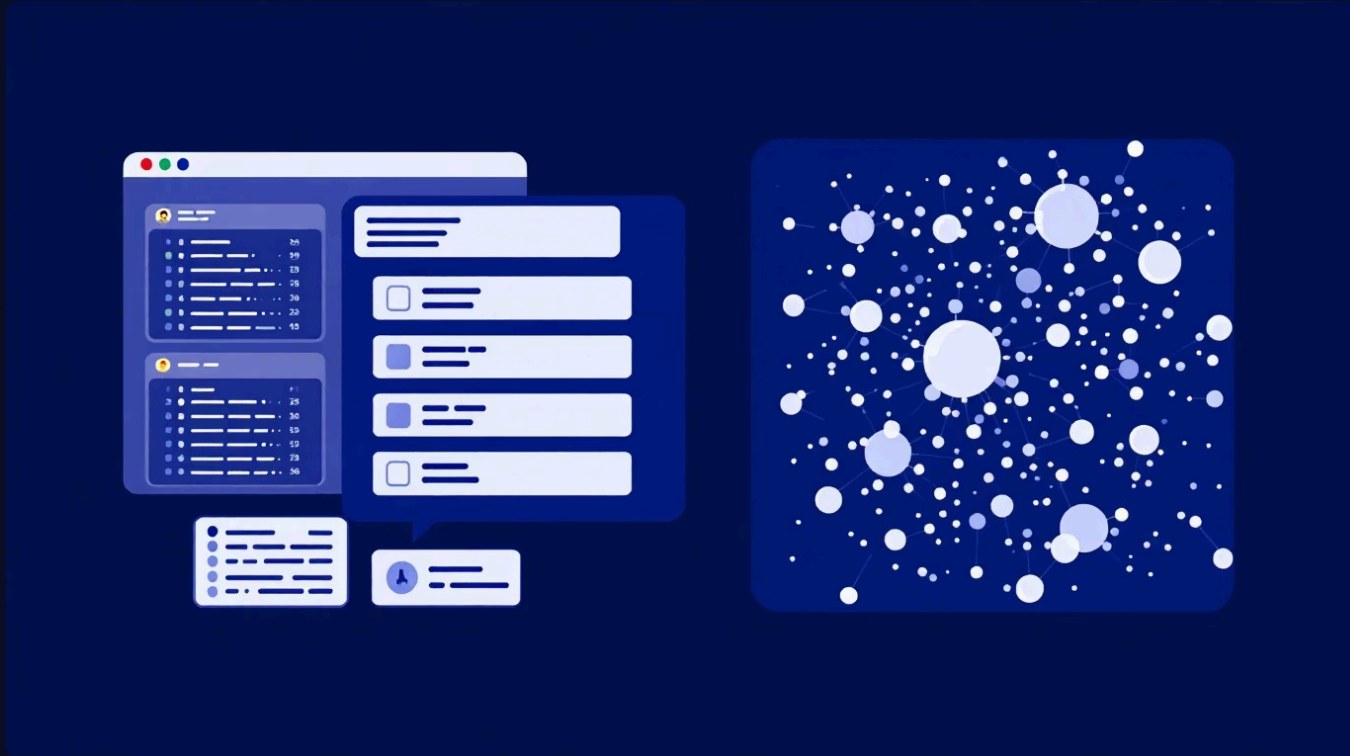
The pipeline converts queries to embeddings, retrieves top-K chunks, composes a context window with citations, and calls the LLM for a grounded answer. If retrieval fails, the system returns a safe fallback message.



Core Components

- Document Processor – extraction, OCR, chunking
- Embedding Service – batch embeddings, async jobs
- Vector Store (FAISS) – nearest neighbor search
- Retriever Module – relevance ranking, Top-K
- LLM Service – prompt assembly, safety checks
- API Layer – upload, query, monitoring

Data Model & Storage



Relational DB

Holds document metadata, chunk IDs, page refs, upload timestamps, and audit logs for traceability.



Vector Store (FAISS)

Stores dense embeddings and pointers to metadata for ultra-fast semantic search.

Security & Performance

Security

JWT auth, API keys, input validation, file scanning, role-based access planned, and prompt grounding to limit hallucinations.

Performance

Batch embedding, in-memory FAISS, async APIs, caching options (Redis), and target median latency <3s for typical queries.

Monitoring and observability (metrics, logging, tracing) support SLA and debugging for technical stakeholders.

銀月夜城
ROADMAP



Conclusion & Roadmap

Now

Working RAG prototype: ingest, index, retrieve, and generate grounded answers from energy docs.

Next

Integrate Redis caching, move vector store to managed cloud (Pinecone), add RBAC and UI dashboard.

Future

Multi-document filtering, advanced citation scoring, offline analytics, and operational rollout with SLAs.

Outcome: faster, traceable, and context-aware access to critical energy documentation – reducing search time and operational risk.